

Lab No 3: Implementation and Analysis of Sorting Algorithms – Heap Sort (Unit 2)

1. Objective and Real-World Application

Objective: To implement a Max-Heap structure and apply it to execute the Heap Sort algorithm efficiently.

Real-World Application: Heap sort is used in embedded and safety-critical systems (like automotive or aerospace software) because it provides a guaranteed worst-case time complexity of $O(n \log n)$ without requiring additional auxiliary memory ($O(1)$ auxiliary space).

2. Pre-requisites (Micro-topics)

- Binary Tree representation using arrays (parent-child index mappings: $2i+1$, $2i+2$).
- The "Heapify" procedure and building a heap from an unsorted array in $O(n)$ time.
- Properties of Complete Binary Trees.

3. Brief Description and Tools

Step 1: Build a Max-Heap from the given unsorted array.

Step 2: Swap the root element (maximum value) with the last element of the heap.

Step 3: Reduce the heap size by 1 and call heapify on the root node to restore the Max-Heap property.

Step 4: Repeat steps 2-3 until the entire array is sorted.

Tools: Python language only. Editors like VS Code, PyCharm, and Google Colab.

4. Descriptive Questions

Level 1: Understanding and Implementation

Question: Outline the parent-child mathematical index mappings used to represent a complete binary tree inside a flat 1D array, and write out the heapify structural utility procedure.

Level 2: Applying Concepts

Question: Walk through the step-by-step conversion of an unsorted input array [19, 15, 10, 7, 12, 11] into a structural Max-Heap array format using the bottom-up construction approach.

Level 3: Advanced Challenge

Question: Prove that an arbitrary array of size n can be transformed into a valid Max-Heap structure in strict $O(n)$ linear time complexity, despite the heapify function requiring $O(\log n)$ time per invocation.